Hadoop Admin Assignment — Part 1

My lab notes: Docker + Kind + HDFS / YARN / ZK / Kafka / HBase / Solr

Part 1 — Environment Setup (Docker + Kubernetes)

What I did in this part: I set up a small Hadoop ecosystem on Kubernetes using Kind on my lab VM, then brought up HDFS, YARN, ZooKeeper, Kafka, HBase and Solr so they can talk to each other inside the cluster.

My setup

- VM: `10.1.0.213` (hostname `alphtatalakehouse6`)
- Kind cluster name: `hadoop-lab`
- Namespace: `hadoop`
- Working folder: `/opt/hadoop-lab`

I kept all configs and YAML under `/opt/hadoop-lab` so I can find everything later.

How the pieces fit (my understanding)

On the VM I installed Docker. Then I used Kind to create a Kubernetes cluster (1 control-plane + 2 workers). Each Kind "node" is just a Docker container.

Inside that cluster I run my services as Pods. The important thing I learned: **don't hardcode Pod IPs**. Pod IPs change when pods restart. I used Kubernetes Service names instead, like `namenode`, `resourcemanager`, `zk-0.zk-hs.hadoop.svc.cluster.local`.

Rough picture:

```
VM
 -> Docker
    -> Kind nodes
       -> Pods (HDFS, YARN, ZK, Kafka, HBase, Solr)
```

Step 1 — Install Docker

I needed Docker first because Kind and all my service images run as containers.

```
dnf install -y dnf-plugins-core
dnf config-manager --add-repo https://download.docker.com/linux/rhel/docker-ce.repo
dnf install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
systemctl enable --now docker
docker --version
```

To confirm Docker was actually working I ran:

```
docker run --rm hello-world
```

I got the “Hello from Docker!” message, so Docker was fine.

Step 2 — Install kubectl

kubectl is what I use to create pods, check status, apply YAML, etc.

```
cd /tmp
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client
```

Step 3 — Install Kind and create the cluster

Kind = Kubernetes in Docker. I used this instead of Minikube.

```
cd /tmp
curl -Lo kind https://kind.sigs.k8s.io/dl/v0.30.0/kind-linux-amd64
chmod +x kind
install -o root -g root -m 0755 kind /usr/local/bin/kind
```

Cluster config I used (1 control-plane, 2 workers):

```
cat > /root/kind-cluster.yaml <<'EOF'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
EOF

kind create cluster --name hadoop-lab --config /root/kind-cluster.yaml
kubectl get nodes
kubectl cluster-info
```

All 3 nodes came up as Ready.

Then I created the namespace and my project folders:

```
kubectl create namespace hadoop
kubectl get namespaces

mkdir -p /opt/hadoop-lab/{hdfs,yarn,spark,kafka,hbase,solr,zookeeper,scripts,docs}
mkdir -p /opt/hadoop-lab/hdfs/{conf,manifests,data}
```

Note: my VM has about 8 vCPU / 32 GiB RAM. The Kind nodes share that. So I kept resource requests small.

Step 4 — Pull the Docker images I used

I pulled the images on the host first:

```
docker pull apache/hadoop:3.3.6
docker pull zookeeper:3.9
docker pull apache/kafka:3.9.1
docker pull openeuler/hbase:2.6.5-oe2403sp3
docker pull solr:9.8
```

Service	Image I used
HDFS / YARN	apache/hadoop:3.3.6
ZooKeeper	zookeeper:3.9
Kafka	apache/kafka:3.9.1
HBase	openeuler/hbase:2.6.5-oe2403sp3
Solr	solr:9.8

I also kept a simple Dockerfile in each service folder (basically `FROM <image>`) under `/opt/hadoop-lab/*/Dockerfile`.

When pods started, Kind pulled the images as needed (`imagePullPolicy: IfNotPresent`).

Step 5 — HDFS (1 NameNode + 2 DataNodes)

Configs I wrote

Location: `/opt/hadoop-lab/hdfs/conf/`

In `core-site.xml` I set:

- `fs.defaultFS = hdfs://namenode:9000`

I used the Service name `namenode`, not a Pod IP.

In `hdfs-site.xml` I mainly set:

- `dfs.replication = 2` (I only have 2 DataNodes in Part 1; for Part 2 when they ask RF=3 I will add one more DataNode first)
- name/data dirs
- `dfs.namenode.rpc-bind-host = 0.0.0.0` so it listens properly inside the container
- `dfs.namenode.datanode.registration.ip-hostname-check = false` (this avoided hostname mismatch issues in K8s)

Commands I ran to deploy

```
kubectl -n hadoop create configmap hdfs-config \
  --from-file=/opt/hadoop-lab/hdfs/conf/core-site.xml \
  --from-file=/opt/hadoop-lab/hdfs/conf/hdfs-site.xml \
  --dry-run=client -o yaml | kubectl apply -f -

kubectl apply -f /opt/hadoop-lab/hdfs/manifests/
kubectl get pods -n hadoop -o wide
```

How I verified

```
kubectl exec -n hadoop namenode-0 -- hdfs dfsadmin -report
kubectl exec -n hadoop namenode-0 -- hdfs dfs -ls /
```

Result: NameNode Running, **Live datanodes (2)**.

Screenshot: see Appendix → HDFS.

Step 6 — YARN (1 ResourceManager + 2 NodeManagers)

This part gave me some trouble at first.

Problem I hit

If I only put ports in `yarn-site.xml` and don't think about how NodeManagers advertise themselves, NMs don't register properly after restart. Hardcoding a Pod IP is also bad because the IP changes.

What I did instead

1. ResourceManager address uses the Service name: `resourcemanager:8032`
2. RM binds on `0.0.0.0`
3. For NodeManager hostname I used the Pod IP at runtime

In config I kept a placeholder:

```
<property>
  <name>yarn.nodemanager.hostname</name>
  <value>${env.POD_IP}</value>
</property>
```

In the NM pod start command I take `status.podIP` from Kubernetes, replace `${env.POD_IP}`, then start `yarn nodemanager`.

Configs are under `/opt/hadoop-lab/yarn/conf/` (`yarn-site.xml`, `mapred-site.xml`, `capacity-scheduler.xml`).

I also prepared capacity scheduler queues already (production 60%/max 80%, development 40%/max 100%) because Part 3 will need that.

Commands I ran

```
kubectl apply -f /opt/hadoop-lab/yarn/manifests/
kubectl get pods -n hadoop | grep -E 'resourcemanager|nodemanager|jobhistory'
kubectl exec -n hadoop deploy/resourcemanager -- yarn node -list
```

Result: **Total Nodes: 2**, both RUNNING.

Screenshot: see Appendix → YARN.

Step 7 — ZooKeeper (3 nodes)

Kafka, HBase and Solr all need ZooKeeper, so I brought this up before those services.

```
kubectl apply -f /opt/hadoop-lab/zookeeper/manifests/
kubectl rollout status statefulset/zk -n hadoop --timeout=300s
kubectl get pods -n hadoop -l app=zk
```

I wait until all 3 are Ready (`zk-0`, `zk-1`, `zk-2`).

Health check I used:

```
kubectl exec -n hadoop zk-0 -- bash -lc 'echo ruok | nc 127.0.0.1 2181'
```

Expected output: `imok`

DNS names I used later:

- `zk-0.zk-hs.hadoop.svc.cluster.local:2181`
- `zk-1.zk-hs.hadoop.svc.cluster.local:2181`
- `zk-2.zk-hs.hadoop.svc.cluster.local:2181`

Screenshot: see Appendix → ZooKeeper.

Step 8 — Kafka (3 brokers with ZooKeeper)

I deployed Kafka as a StatefulSet with 3 replicas.

Important points in my config:

- `broker.id` comes from the pod number (`kafka-0` → 0, etc.)
- `advertised.listeners` uses stable DNS like `kafka-0.kafka-hs.hadoop.svc.cluster.local:9092`
- `zookeeper.connect` points to the 3 ZK pods

Commands:

```
kubectl apply -f /opt/hadoop-lab/kafka/manifests/
kubectl get pods -n hadoop -l app=kafka
kubectl logs -n hadoop kafka-0 --tail=50
```

In the logs I checked for Kafka started successfully (ZK mode). Then I also checked brokers like this:

```
kubectl exec -n hadoop kafka-0 -- \
  /opt/kafka/bin/kafka-broker-api-versions.sh \
  --bootstrap-server kafka-0.kafka-hs.hadoop.svc.cluster.local:9092
```

All 3 brokers (`kafka-0`, `kafka-1`, `kafka-2`) are Running.

Screenshot: see Appendix → Kafka.

Step 9 — HBase (1 Master + 2 RegionServers on HDFS)

HBase needs HDFS for storage and ZooKeeper for coordination.

In `hbase-site.xml` I set:

- `hbase.rootdir = hdfs://namenode:9000/hbase`
- ZK quorum = the 3 ZK DNS names
- `hbase.master.hostname = hbase-master` (Service name)

One issue I faced: at first the master registered with the pod hostname (`hbase-master-xxxxx`), and RegionServers couldn't resolve that name. After I forced `hbase.master.hostname=hbase-master`, the RegionServers connected fine.

Also for each RegionServer I set hostname to:

`hbase-rs-0.hbase-rs.hadoop.svc.cluster.local` (and same pattern for rs-1).

Commands:

```
kubectl -n hadoop create configmap hbase-config \
  --from-file=/opt/hadoop-lab/hbase/conf/hbase-site.xml \
  --from-file=/opt/hadoop-lab/hdfs/conf/core-site.xml \
  --from-file=/opt/hadoop-lab/hdfs/conf/hdfs-site.xml \
  --dry-run=client -o yaml | kubectl apply -f -

kubectl -n hadoop create configmap hbase-scripts \
  --from-file=/opt/hadoop-lab/hbase/scripts/start-master.sh \
  --from-file=/opt/hadoop-lab/hbase/scripts/start-rs.sh \
  --dry-run=client -o yaml | kubectl apply -f -

kubectl apply -f /opt/hadoop-lab/hbase/manifests/
kubectl get pods -n hadoop | grep hbase
```

Verify:

```
kubectl exec -n hadoop deploy/hbase-master -- bash -lc 'echo -e "status\nexit" | hbase shell -n'
```

Result I got:

```
1 active master, 0 backup masters, 2 servers, 0 dead, 1.0000 average load
```

Screenshot: see Appendix → HBase.

Step 10 — Solr (2-node SolrCloud with ZooKeeper)

For SolrCloud I pointed both nodes to the same ZK ensemble.

One issue: Kubernetes was injecting env vars like `SOLR_PORT=tcp://...` because I had a Service named `solr`. That broke the Solr start script. I fixed it by setting `enableServiceLinks: false` on the Solr pods and starting with `--force` (needed when running as root).

Commands I ran:

```
kubectl -n hadoop create configmap solr-scripts \
  --from-file=/opt/hadoop-lab/solr/scripts/start-solr.sh \
  --dry-run=client -o yaml | kubectl apply -f -

kubectl apply -f /opt/hadoop-lab/solr/manifests/
kubectl get pods -n hadoop -l app=solr
kubectl get pods -n hadoop -l app=solr -o wide
```

Both `solr-0` and `solr-1` came to Ready `1/1`.

(Creating collection / schema / queries is for Part 7. Here I only needed SolrCloud up.)

Screenshot: see Appendix → Solr.

Step 11 — Bring the whole stack up (commands I use)

I don't rely on a wrapper script for submission. These are the exact commands, in order.

First make sure the namespace exists:

```
kubectl get ns hadoop || kubectl create namespace hadoop
```

Refresh configmaps if I changed XML/scripts:

```
kubectl -n hadoop create configmap hdfs-config \
  --from-file=/opt/hadoop-lab/hdfs/conf/core-site.xml \
  --from-file=/opt/hadoop-lab/hdfs/conf/hdfs-site.xml \
  --dry-run=client -o yaml | kubectl apply -f -

kubectl -n hadoop create configmap yarn-config \
  --from-file=/opt/hadoop-lab/yarn/conf/ \
  --dry-run=client -o yaml | kubectl apply -f -

kubectl -n hadoop create configmap hbase-config \
  --from-file=/opt/hadoop-lab/hbase/conf/hbase-site.xml \
  --from-file=/opt/hadoop-lab/hdfs/conf/core-site.xml \
  --from-file=/opt/hadoop-lab/hdfs/conf/hdfs-site.xml \
  --dry-run=client -o yaml | kubectl apply -f -

kubectl -n hadoop create configmap hbase-scripts \
  --from-file=/opt/hadoop-lab/hbase/scripts/start-master.sh \
  --from-file=/opt/hadoop-lab/hbase/scripts/start-rs.sh \
  --dry-run=client -o yaml | kubectl apply -f -

kubectl -n hadoop create configmap solr-scripts \
  --from-file=/opt/hadoop-lab/solr/scripts/start-solr.sh \
  --dry-run=client -o yaml | kubectl apply -f -
```

Then apply manifests service by service (order matters — ZK before Kafka/HBase/Solr):

```
kubectl apply -f /opt/hadoop-lab/hdfs/manifests/
kubectl apply -f /opt/hadoop-lab/yarn/manifests/
kubectl apply -f /opt/hadoop-lab/zookeeper/manifests/
kubectl rollout status statefulset/zk -n hadoop --timeout=300s
kubectl apply -f /opt/hadoop-lab/kafka/manifests/
kubectl apply -f /opt/hadoop-lab/hbase/manifests/
kubectl apply -f /opt/hadoop-lab/solr/manifests/
```

Check everything:

```
kubectl get pods,svc -n hadoop -o wide
```

Step 12 — Final checks I ran (with outputs)

Full pod list

```
kubectl get pods -n hadoop -o wide
```

All main pods are Running:

- namenode-0, datanode-0, datanode-1
- resourcemanager, 2 nodemanagers, jobhistory
- zk-0, zk-1, zk-2
- kafka-0, kafka-1, kafka-2
- hbase-master, hbase-rs-0, hbase-rs-1
- solr-0, solr-1

HDFS

```
kubectl exec -n hadoop namenode-0 -- hdfs dfsadmin -report
```

→ Live datanodes (2)

YARN

```
kubectl exec -n hadoop deploy/resourcemanager -- yarn node -list
```

→ Total Nodes: 2 (RUNNING)

ZooKeeper

```
kubectl exec -n hadoop zk-0 -- bash -lc 'echo ruok | nc 127.0.0.1 2181'
```

→ imok

Kafka

```
kubectl get pods -n hadoop -l app=kafka
```

→ kafka-0/1/2 Running

HBase

```
kubectl exec -n hadoop deploy/hbase-master -- bash -lc 'echo -e "status\nexit" | hbase shell -n'
```

→ 1 active master, 2 servers

Solr

```
kubectl get pods -n hadoop -l app=solr
```

→ solr-0 and solr-1 Ready 1/1

Deliverables for Part 1 (what was asked)

Asked for	**What I prepared**
Dockerfiles for HDFS, YARN, HBase, Kafka, Solr	/opt/hadoop-lab/*/Dockerfile
Kubernetes manifests (StatefulSets / Deployments / Services / ConfigMaps)	/opt/hadoop-lab/*/manifests/
Services can communicate inside the cluster	Using Service DNS names (shown in configs + running status)
Steps to bring cluster up	Commands in Step 11 above
Documentation + screenshots + commands	This document + Appendix screenshots

Config files are also in:

- `/opt/hadoop-lab/hdfs/conf/`
- `/opt/hadoop-lab/yarn/conf/`
- `/opt/hadoop-lab/zookeeper/conf/`
- `/opt/hadoop-lab/kafka/conf/`
- `/opt/hadoop-lab/hbase/conf/`
- `/opt/hadoop-lab/solr/conf/`

Small issues I faced (what I changed, with paths)

1) YARN / NodeManager registration

Problem: NodeManagers were not registering cleanly after restart if hostname/IP was wrong.

Files I changed:

- `/opt/hadoop-lab/yarn/conf/yarn-site.xml`
- `/opt/hadoop-lab/yarn/manifests/nodemanager.yaml`

What I added / set in `yarn-site.xml`:

```
<!-- ResourceManager uses Service DNS, not Pod IP -->
<property>
  <name>yarn.resourcemanager.hostname</name>
  <value>resourcemanager</value>
</property>
<property>
  <name>yarn.resourcemanager.address</name>
  <value>resourcemanager:8032</value>
</property>
<property>
  <name>yarn.resourcemanager.bind-host</name>
  <value>0.0.0.0</value>
</property>

<!-- NodeManager hostname filled at runtime from Pod IP -->
<property>
  <name>yarn.nodemanager.hostname</name>
  <value>${env.POD_IP}</value>
</property>
<property>
  <name>yarn.nodemanager.bind-host</name>
  <value>0.0.0.0</value>
</property>
```

What I added in `nodemanager.yaml`:

- env `POD_IP` from `status.podIP`
- start script does `sed` to replace `${env.POD_IP}` with the real IP, then starts `yarn nodemanager`

---

2) HBase RegionServers couldn't find master

Problem: Master registered in ZK using pod name like `hbase-master-xxxxx`, which other pods cannot resolve.

Files I changed:

- `/opt/hadoop-lab/hbase/conf/hbase-site.xml`
- `/opt/hadoop-lab/hbase/scripts/start-rs.sh`
- `/opt/hadoop-lab/hbase/manifests/hbase.yaml` (uses those files via ConfigMaps)

What I added in `hbase-site.xml`:

```
<property>
  <name>hbase.master.hostname</name>
  <value>hbase-master</value>
</property>
<property>
  <name>hbase.master.ipc.address</name>
  <value>0.0.0.0</value>
</property>
<property>
  <name>hbase.zookeeper.quorum</name>
  <value>zk-0.zk-hs.hadoop.svc.cluster.local,zk-1.zk-hs.hadoop.svc.cluster.local,zk-2.zk-
hs.hadoop.svc.cluster.local</value>
</property>
```

What I added in `start-rs.sh` (per RegionServer):

```
HOSTNAME_FQDN="${POD_NAME}.hbase-rs.hadoop.svc.cluster.local"
# then inject into hbase-site.xml:
# hbase.regionserver.hostname = hbase-rs-0.hbase-rs.hadoop.svc.cluster.local
# (same pattern for hbase-rs-1)
```

After this, `status` showed: `1 active master ... 2 servers`.

---

3) Solr crash loop

Problem: Because I had a Service named `solr`, Kubernetes injected env like `SOLR_PORT=tcp://10.x.x.x:8983`. Solr start script treated that as a port number and crashed.

Files I changed:

- `/opt/hadoop-lab/solr/manifests/solr.yaml`
- `/opt/hadoop-lab/solr/scripts/start-solr.sh`

What I added in `solr.yaml`:

```
spec:
  template:
    spec:
      enableServiceLinks: false   # stop bad SOLR_PORT injection
```

What I added / used in `start-solr.sh`:

```
unset SOLR_PORT SOLR_SERVICE_PORT SOLR_PORT_8983_TCP || true
export SOLR_HOST="${POD_NAME}.solr-hs.hadoop.svc.cluster.local"
export SOLR_PORT=8983
exec solr-fg -cloud -z "$ZK_HOST" -p 8983 --force
```

(`--force` is needed because the container runs as root.)

---

4) Replication factor (Part 1 vs Part 2)

File: `/opt/hadoop-lab/hdfs/conf/hdfs-site.xml`

What I set for Part 1 (only 2 DataNodes):

```
<property>
  <name>dfs.replication</name>
  <value>2</value>
</property>
```

For Part 2 (RF=3) I will first scale DataNodes to 3, then change this to `3` and run `hdfs dfs -setrep -R 3 /...`.

Appendix — Screenshots

I attached the screenshots from my running cluster below (same files under `docs/screenshots/`).

Appendix — Screenshots from my running cluster

These are from my Kind cluster after everything came up. I kept the raw command output as well.

Full cluster — pods and services

cluster-status.txt

```
===== kubectl get pods -n hadoop -o wide =====
NAME                              READY   STATUS      RESTARTS      AGE     IP            NODE                NOMINATED NODE   READINESS GATES
datanode-0                        1/1     Running     0             21h     10.244.2.6    hadoop-lab-worker2  <none>           <none>
datanode-1                        1/1     Running     0             21h     10.244.1.10   hadoop-lab-worker   <none>           <none>
hbase-master-77c899f866-545d5     1/1     Running     0             3m20s   10.244.2.32   hadoop-lab-worker2  <none>           <none>
hbase-rs-0                        1/1     Running     0             3m20s   10.244.1.26   hadoop-lab-worker   <none>           <none>
hbase-rs-1                        1/1     Running     0             3m19s   10.244.2.33   hadoop-lab-worker2  <none>           <none>
jobhistory-6c4f75fd9d-5vf1h       1/1     Running     0             11h     10.244.2.10   hadoop-lab-worker2  <none>           <none>
kafka-0                           1/1     Running     0             14m     10.244.2.25   hadoop-lab-worker2  <none>           <none>
kafka-1                           1/1     Running     0             14m     10.244.1.23   hadoop-lab-worker   <none>           <none>
kafka-2                           1/1     Running     0             14m     10.244.2.26   hadoop-lab-worker2  <none>           <none>
namenode-0                        1/1     Running     0             21h     10.244.1.9    hadoop-lab-worker   <none>           <none>
nodemanager-dfbcdcdbc-dzg7m       1/1     Running     0             9h      10.244.2.16   hadoop-lab-worker2  <none>           <none>
nodemanager-dfbcdcdbc-v9fsr       1/1     Running     0             9h      10.244.1.18   hadoop-lab-worker   <none>           <none>
pvc-debug                         0/1     Completed   0             22h     10.244.2.4    hadoop-lab-worker2  <none>           <none>
pvc-debug-1                       0/1     Completed   0             21h     10.244.1.7    hadoop-lab-worker   <none>           <none>
resourcemanager-76c7b7d9bc-nbrgh  1/1     Running     1 (9h ago)    11h     10.244.1.12   hadoop-lab-worker   <none>           <none>
solr-0                            1/1     Running     0             3m15s   10.244.2.34   hadoop-lab-worker2  <none>           <none>
solr-1                            1/1     Running     0             2m10s   10.244.1.28   hadoop-lab-worker   <none>           <none>
zk-0                              1/1     Running     0             15m     10.244.2.21   hadoop-lab-worker2  <none>           <none>
zk-1                              1/1     Running     0             15m     10.244.2.22   hadoop-lab-worker2  <none>           <none>
zk-2                              1/1     Running     0             15m     10.244.1.21   hadoop-lab-worker   <none>           <none>

===== kubectl get svc -n hadoop =====
NAME             TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)                                AGE
datanode         ClusterIP   None            <none>        9866/TCP,9864/TCP                      23h
hbase-master     ClusterIP   10.96.47.21     <none>        16000/TCP,16010/TCP                    13m
hbase-rs         ClusterIP   None            <none>        16020/TCP,16030/TCP                    13m
jobhistory       ClusterIP   10.96.114.145   <none>        10020/TCP,19888/TCP                    11h
kafka            ClusterIP   10.96.248.84    <none>        9092/TCP                               14m
kafka-hs         ClusterIP   None            <none>        9092/TCP                               14m
namenode         ClusterIP   10.96.60.12     <none>        9000/TCP,9870/TCP                      23h
resourcemanager  ClusterIP   10.96.158.157   <none>        8032/TCP,8030/TCP,8031/TCP,8088/TCP    11h
solr             ClusterIP   10.96.34.104    <none>        8983/TCP                               13m
solr-hs          ClusterIP   None            <none>        8983/TCP                               13m
zk-hs            ClusterIP   None            <none>        2181/TCP,2888/TCP,3888/TCP             15m
zookeeper        ClusterIP   10.96.202.67    <none>        2181/TCP                               15m
```

HDFS — hdfs dfsadmin -report

dfsadmin-report.txt

```
===== hdfs dfsadmin -report =====
Configured Capacity: 131357212672 (122.34 GB)
Present Capacity: 71162984862 (66.28 GB)
DFS Remaining: 71159505906 (66.27 GB)
DFS Used: 3478956 (3.32 MB)
DFS Used%: 0.00%
Replicated Blocks:
Under replicated blocks: 6
Blocks with corrupt replicas: 0
Missing blocks: 0
Missing blocks (with replication factor 1): 0
Low redundancy blocks with highest priority to recover: 0
Pending deletion blocks: 0
Erasure Coded Block Groups:
Low redundancy block groups: 0
Block groups with corrupt internal blocks: 0
Missing block groups: 0
Low redundancy blocks with highest priority to recover: 0
Pending deletion blocks: 0

-------------------------------------------------
Live datanodes (2):

Name: 10.244.1.10:9866 (datanode-1.datanode.hadoop.svc.cluster.local)
Hostname: datanode-1.datanode.hadoop.svc.cluster.local
Decommission Status : Normal
Configured Capacity: 65678606336 (61.17 GB)
DFS Used: 1739478 (1.66 MB)
Non DFS Used: 29962896682 (27.91 GB)
DFS Remaining: 35579752953 (33.14 GB)
DFS Used%: 0.00%
DFS Remaining%: 54.17%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 2
Last contact: Thu Oct 15 14:31:40 UTC 2026
Last Block Report: Thu Oct 15 12:19:29 UTC 2026
Num of Blocks: 23


Name: 10.244.2.6:9866 (datanode-0.datanode.hadoop.svc.cluster.local)
Hostname: datanode-0.datanode.hadoop.svc.cluster.local
Decommission Status : Normal
Configured Capacity: 65678606336 (61.17 GB)
DFS Used: 1739478 (1.66 MB)
Non DFS Used: 29962896682 (27.91 GB)
DFS Remaining: 35579752953 (33.14 GB)
DFS Used%: 0.00%
DFS Remaining%: 54.17%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 2
Last contact: Thu Oct 15 14:31:43 UTC 2026
Last Block Report: Thu Oct 15 11:05:19 UTC 2026
Num of Blocks: 23
```

YARN — yarn node -list

yarn-node-list.txt

```
===== yarn node -list =====
2026-10-15 14:31:49 INFO  DefaultNoHARMFailoverProxyProvider:64 - Connecting to ResourceManager at resourcemanager/10.96.158.157:8032
Total Nodes:2
         Node-Id     Node-State Node-Http-Address Number-of-Running-Containers
nodemanager-dfbcdcdbc-v9fsr:42251        RUNNING nodemanager-dfbcdcdbc-v9fsr:8042                    0
nodemanager-dfbcdcdbc-dzg7m:39751        RUNNING nodemanager-dfbcdcdbc-dzg7m:8042                    0
```

ZooKeeper — ruok check

zk-ruok.txt

```
===== zookeeper ruok =====
zk-0: imok
zk-1: imok
zk-2: imok
```

Kafka — broker check

brokers.txt

```
===== kafka broker api versions (broker 0) =====
kafka-1.kafka-hs.hadoop.svc.cluster.local:9092 (id: 1 rack: null) -> (
Produce(0): 0 to 11 [usable: 11],
Fetch(1): 0 to 17 [usable: 17],
ListOffsets(2): 0 to 9 [usable: 9],
Metadata(3): 0 to 12 [usable: 12],
LeaderAndIsr(4): 0 to 7 [usable: 7],
StopReplica(5): 0 to 4 [usable: 4],
UpdateMetadata(6): 0 to 8 [usable: 8],
ControlledShutdown(7): 0 to 3 [usable: 3],
OffsetCommit(8): 0 to 9 [usable: 9],
OffsetFetch(9): 0 to 9 [usable: 9],
FindCoordinator(10): 0 to 6 [usable: 6],
JoinGroup(11): 0 to 9 [usable: 9],
Heartbeat(12): 0 to 4 [usable: 4],
LeaveGroup(13): 0 to 5 [usable: 5],
SyncGroup(14): 0 to 5 [usable: 5],
DescribeGroups(15): 0 to 5 [usable: 5],
ListGroups(16): 0 to 5 [usable: 5],
SaslHandshake(17): 0 to 1 [usable: 1],
ApiVersions(18): 0 to 4 [usable: 4],
CreateTopics(19): 0 to 7 [usable: 7],
DeleteTopics(20): 0 to 6 [usable: 6],
DeleteRecords(21): 0 to 2 [usable: 2],
InitProducerId(22): 0 to 5 [usable: 5],
OffsetForLeaderEpoch(23): 0 to 4 [usable: 4],
AddPartitionsToTxn(24): 0 to 5 [usable: 5],
AddOffsetsToTxn(25): 0 to 4 [usable: 4],
EndTxn(26): 0 to 4 [usable: 4],
WriteTxnMarkers(27): 0 to 1 [usable: 1],
TxnOffsetCommit(28): 0 to 4 [usable: 4],
DescribeAcls(29): 0 to 3 [usable: 3],
CreateAcls(30): 0 to 3 [usable: 3],
DeleteAcls(31): 0 to 3 [usable: 3],
DescribeConfigs(32): 0 to 4 [usable: 4],
AlterConfigs(33): 0 to 2 [usable: 2],
AlterReplicaLogDirs(34): 0 to 2 [usable: 2],
DescribeLogDirs(35): 0 to 4 [usable: 4],
SaslAuthenticate(36): 0 to 2 [usable: 2],
CreatePartitions(37): 0 to 3 [usable: 3],
CreateDelegationToken(38): 0 to 3 [usable: 3],
```

HBase — status

status.txt

```
hbase:001:0> status
1 active master, 0 backup masters, 0 servers, 0 dead, 0.0000 average load
Took 3.5284 seconds
hbase:002:0>
```

Solr — pods Ready

status.txt

```
===== solr status =====
NAME     READY   STATUS    RESTARTS   AGE    IP            NODE                NOMINATED NODE   READINESS GATES
solr-0   1/1     Running   0          5m7s   10.244.2.34   hadoop-lab-worker2  <none>           <none>
solr-1   1/1     Running   0          4m2s   10.244.1.28   hadoop-lab-worker   <none>           <none>
{
  "responseHeader":{
    "status":0,
    "QTime":67
  },
  "cluster":{
    "live_nodes":["solr-0.solr-hs.hadoop.svc.cluster.local:8983_solr","solr-1.solr-hs.hadoop.svc.cluster.local:8983_solr"],
    "collections":{ },
    "properties":{ },
    "roles":{ }
  }
}
```